Лабораторна робота №4

Реалізація підсистеми безпеки, автентифікації та авторизації

Студент: Бабенко Олександр Олександрович

Група: 402-ТН

Викладач: Тютюнник Петро Богданович

1. Посилання на Git-репозиторій

https://github.com/Radonskiy/TRVD_2026_402TN_Babenko_Labs

У ході виконання лабораторної роботи було реалізовано систему автентифікації та авторизації для REST API, розробленого у попередній лабораторній роботі.

Було додано модель користувача (User), яка містить email, пароль та роль (USER або ADMIN). Для забезпечення безпеки паролі користувачів хешуються за допомогою бібліотеки bcrypt перед збереженням у базу даних.

Реалізовано ендпоінти:

- /auth/register — для реєстрації користувача

- /auth/login — для входу та отримання JWT токена

Після успішного логіну користувач отримує JWT токен, який використовується для доступу до захищених маршрутів.

Було реалізовано Guards:

- JwtAuthGuard — перевіряє наявність і валідність токена

- RolesGuard — перевіряє роль користувача

Доступ до ресурсів розмежовано:

- публічні маршрути — доступні всім

- захищені — лише авторизованим користувачам

- адміністративні — лише користувачам з роллю ADMIN

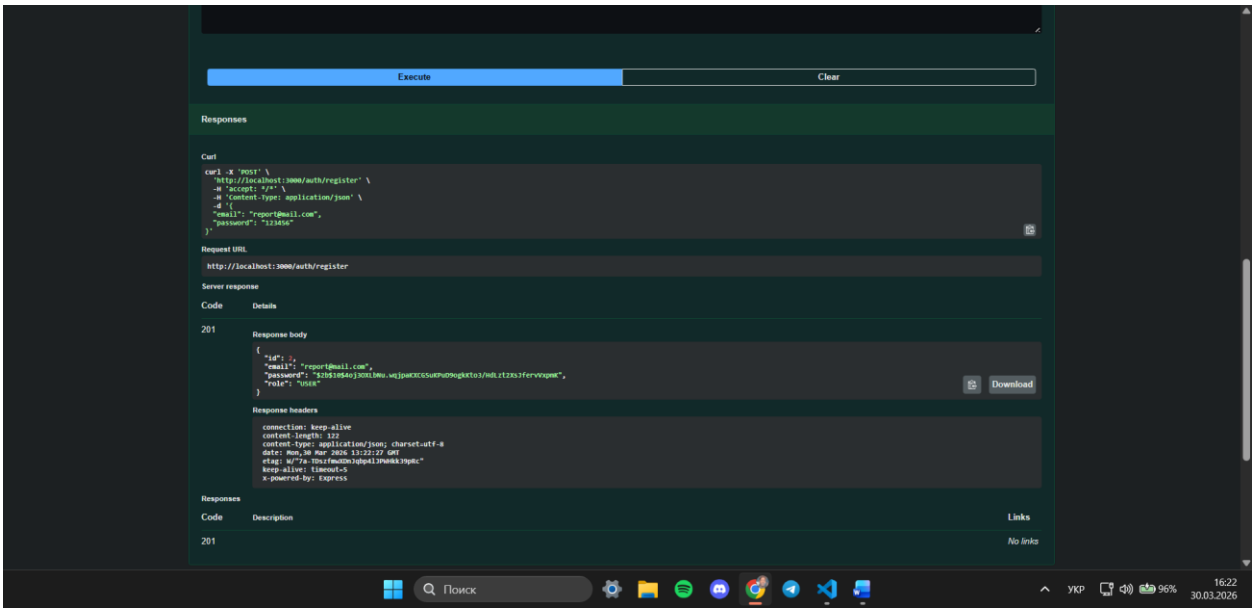

Рисунок 1 – Успішна реєстрація користувача через Swagger (статус 201 Created)

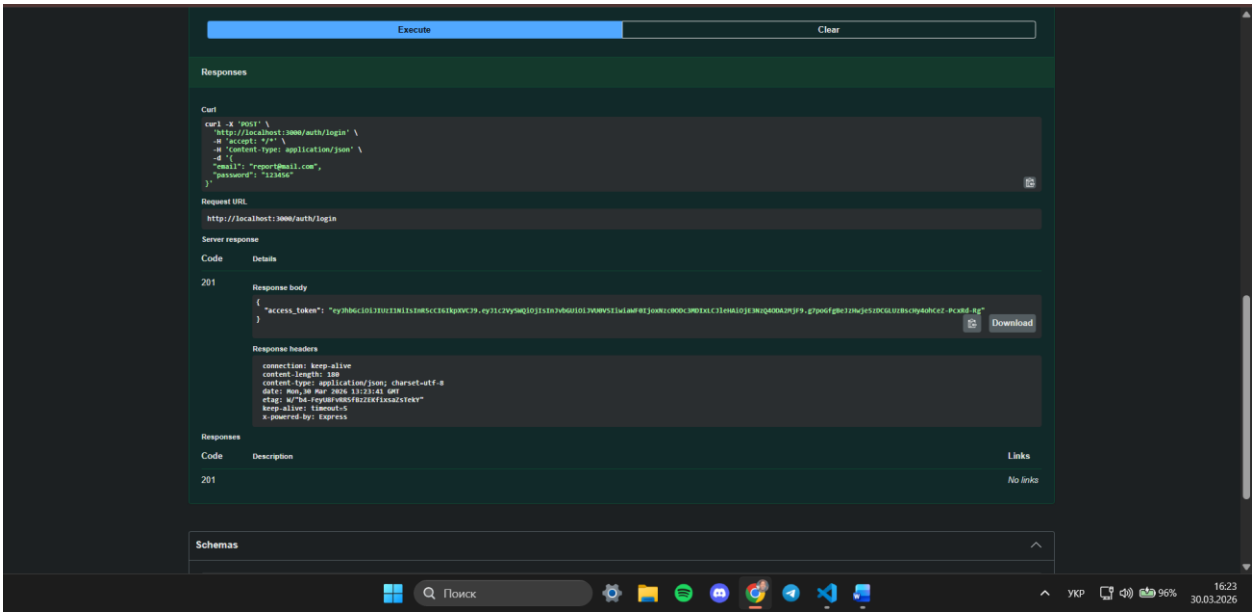

Рисунок 2 – Успішний логін користувача та отримання JWT токена

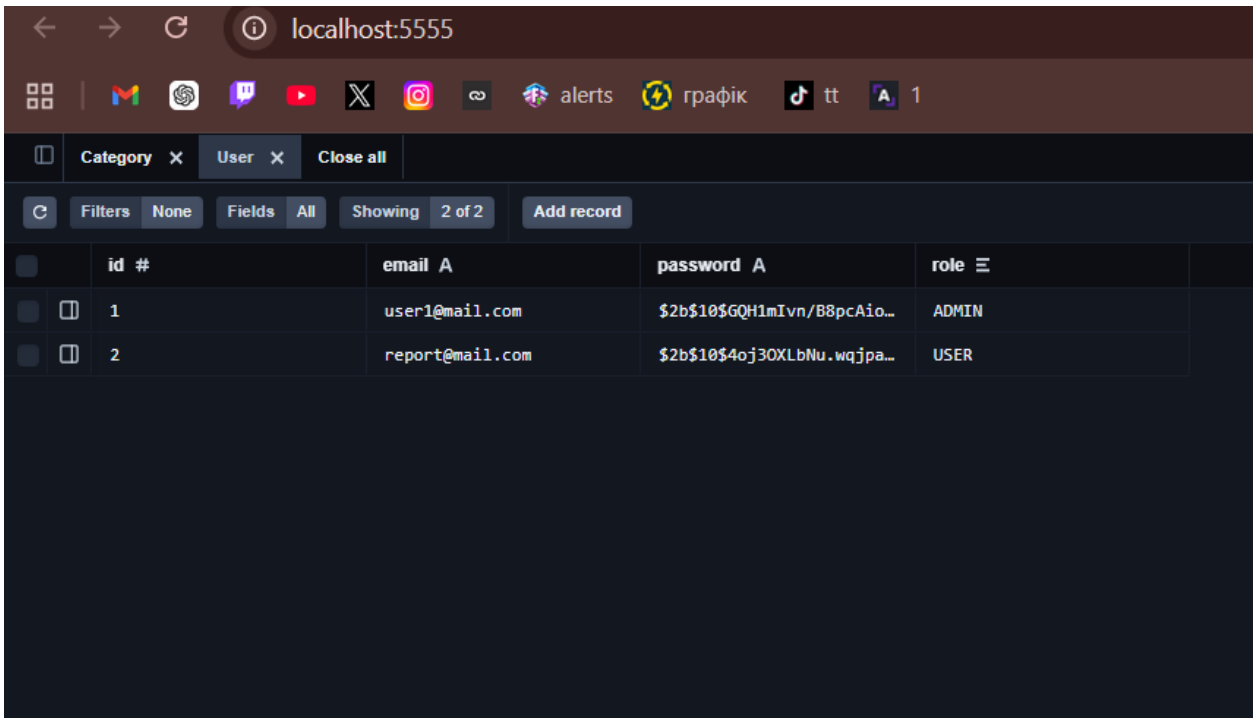

Рисунок 3 – Збереження пароля у вигляді хешу в базі даних (Prisma Studio)

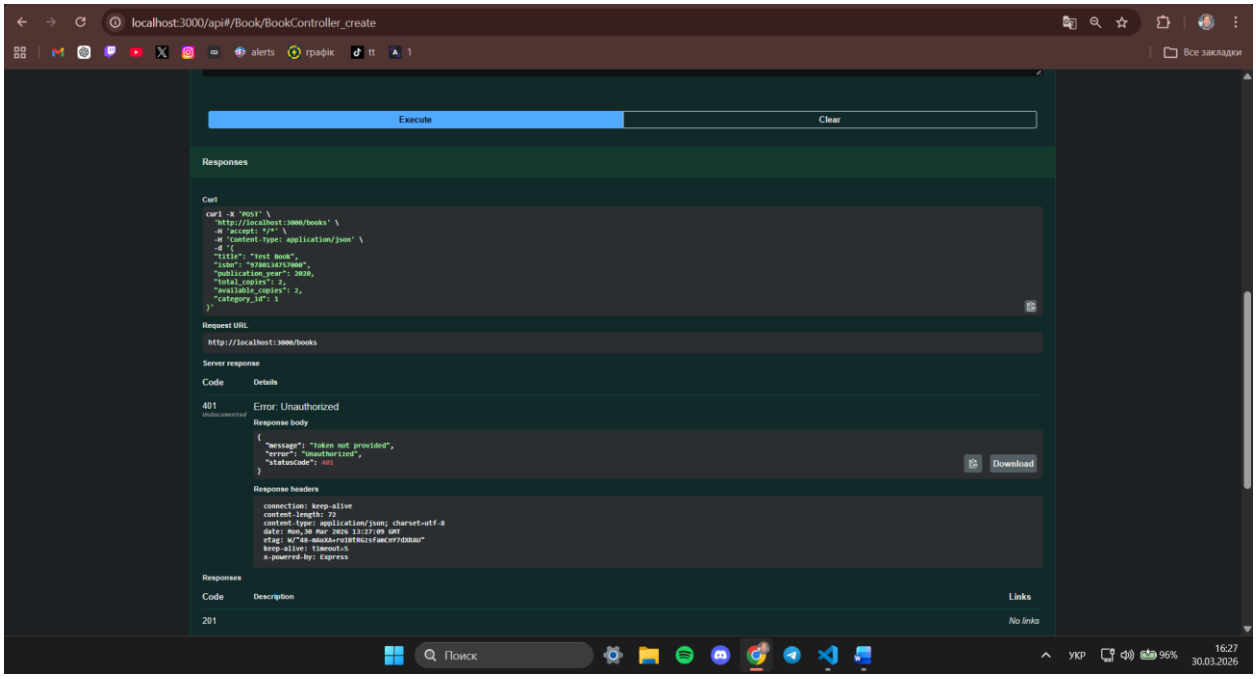

Рисунок 4 – Спроба доступу без токена (401 Unauthorized)

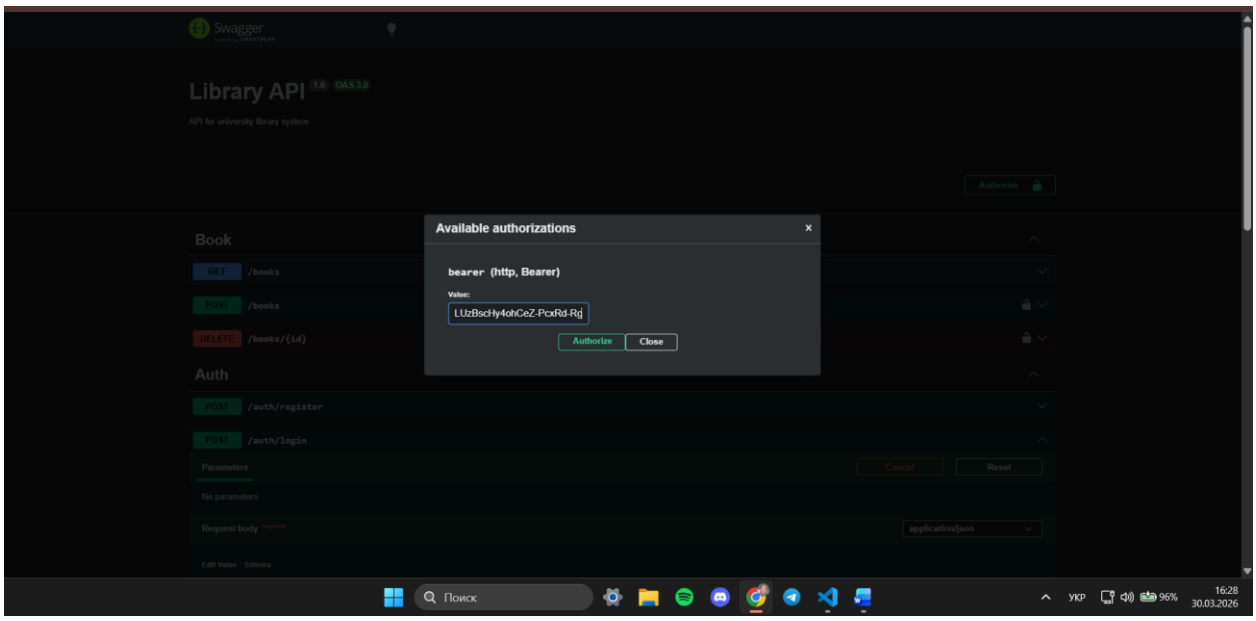

Рисунок 5 – Авторизація з токеном

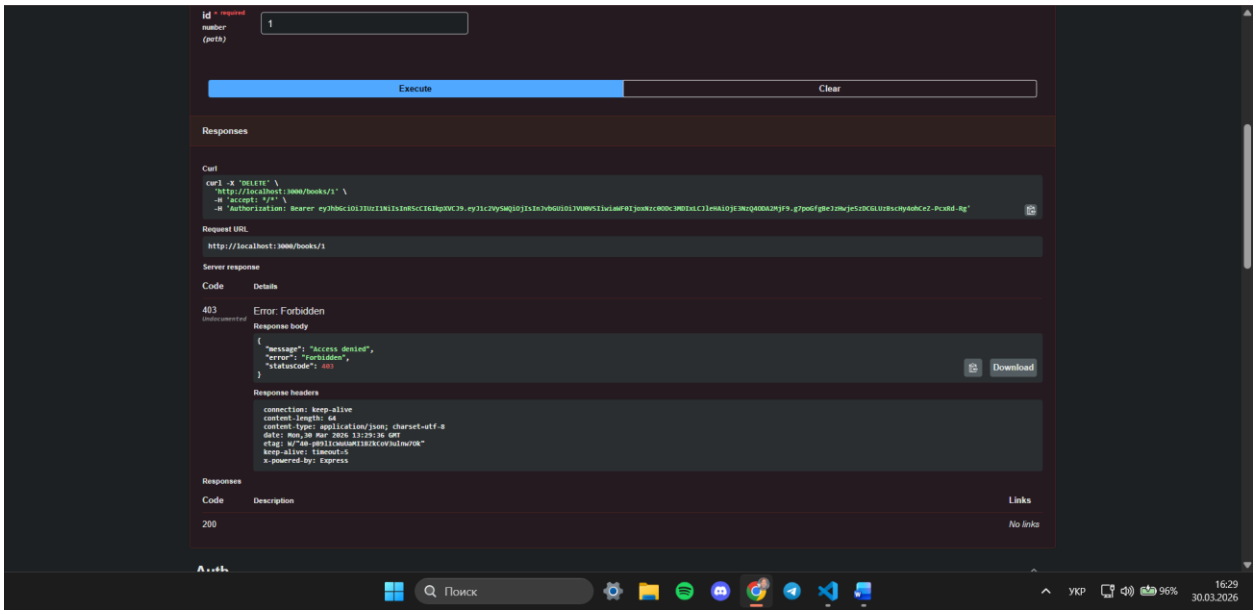

Рисунок 6 – Спроба доступу користувача без прав адміністратора (403 Forbidden)

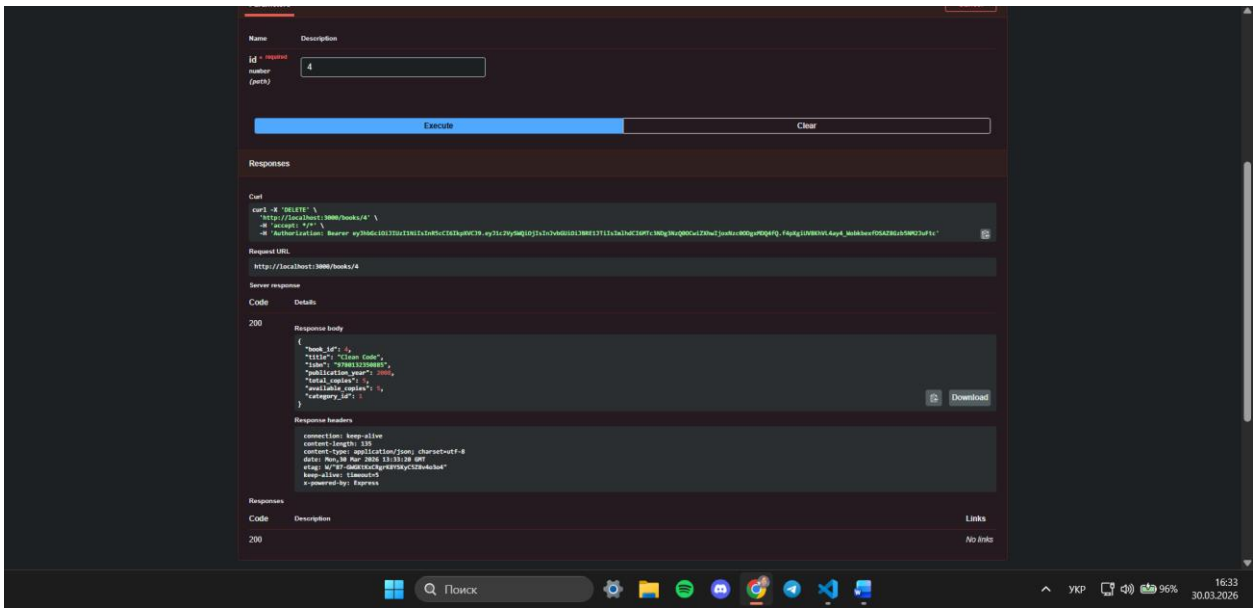

Рисунок 6 – Успішний доступ адміністратора до захищеного ресурсу (200 OK)

Контрольні запитання

1. У чому різниця між автентифікацією (Authentication) та авторизацією (Authorization)?

Автентифікація — це процес перевірки особи користувача. Вона відповідає на питання: «Хто ти?». Наприклад, введення логіна та пароля при вході в систему.

Авторизація — це процес визначення прав доступу користувача. Вона відповідає на питання: «Що тобі дозволено робити?». Наприклад, чи може користувач видаляти записи або тільки переглядати їх.

Таким чином, спочатку виконується автентифікація, а після неї — авторизація.

---

2. Чому не можна зберігати паролі у відкритому вигляді? Що таке "Сіль" (Salt) і навіщо вона потрібна?

Зберігання паролів у відкритому вигляді є небезпечним, оскільки у випадку витоку бази даних зловмисники одразу отримають доступ до всіх облікових записів.

Тому паролі зберігаються у вигляді хешів — результату криптографічної функції, яку неможливо легко відновити.

"Сіль" (Salt) — це випадковий набір символів, який додається до пароля перед хешуванням. Вона потрібна для того, щоб:

- запобігти однаковим хешам для однакових паролів;
- ускладнити атаки типу Rainbow Table.

---

3. Опишіть структуру JWT токена (Header, Payload, Signature). Де зберігається роль користувача?

JWT (JSON Web Token) складається з трьох частин:

- **Header** — містить тип токена та алгоритм підпису (наприклад, HS256).
- **Payload** — містить дані (claims), такі як userId, роль користувача (role), час створення (iat) та час життя (exp).
- **Signature** — криптографічний підпис, який гарантує цілісність токена.

Інформація про роль користувача зберігається в частині **Payload**.

---

4. Яка різниця між симетричним та асиметричним шифруванням при підписі токенів?

Симетричне шифрування використовує один і той самий секретний ключ для створення та перевірки підпису токена (наприклад, алгоритм HS256).

Асиметричне шифрування використовує пару ключів:

- приватний ключ — для підпису;
- публічний ключ — для перевірки.

Симетричне швидше і простіше, але менш безпечне у розподілених системах. Асиметричне безпечніше, але складніше в реалізації.

---

5. Що таке атаки Brute Force та Rainbow Table? Як захиститися від них?

Brute Force — це атака, при якій зловмисник перебирає всі можливі варіанти пароля.

Rainbow Table — це атака з використанням попередньо обчислених таблиць хешів для швидкого знаходження паролів.

Захист:

- використання хешування (bcrypt, argon2);
- додавання "солі" (salt);
- обмеження кількості спроб входу;
- використання складних паролів.

---

6. Поясніть принцип роботи Role-Based Access Control (RBAC)

RBAC — це модель контролю доступу, при якій права визначаються ролями користувачів.

Кожному користувачу призначається роль (наприклад, USER або ADMIN). Кожна роль має набір дозволених дій.

При обробці запиту система:

1. перевіряє автентифікацію (чи є токен);
2. отримує роль користувача з токена;
3. перевіряє, чи має ця роль доступ до ресурсу.

Якщо роль не має доступу — повертається помилка 403 Forbidden.

Висновок

У ході виконання лабораторної роботи було реалізовано підсистему безпеки для веб-API. Було додано механізм реєстрації користувачів із безпечним зберіганням паролів у вигляді хешів. Реалізовано автентифікацію за допомогою JWT токенів, які дозволяють ідентифікувати користувача при кожному запиті.

Також було впроваджено авторизацію на основі ролей (RBAC), що забезпечує розмежування доступу до ресурсів. Захищені маршрути перевіряють наявність токена та права користувача, що дозволяє запобігти несанкціонованому доступу.

У результаті система стала безпечною, масштабованою та готовою до використання в реальних умовах.